

Agile Inception Deck

MADES

Fecha: 26/02/2026

Tabla de contenido

Historial de Versiones.....	2
Información del Proyecto.....	3
Aprobaciones.....	3
Why are we here?.....	3
Impact mapping.....	4
Elevator pitch.....	4
Product Box.....	5
Create a NOT list.....	6
Meet your neighbors.....	7
Show your solution.....	7
Ask what keeps us up at night?.....	11
Size it up.....	11
Complejidad del proyecto.....	12
Be clear on what's going to give.....	13
Trade-Off Slider del Proyecto.....	13
Prioridad Estratégica.....	13
Show what it's going to take.....	15

Historial de Versiones

Fecha	Versión	Autor	Organización	Descripción

--	--	--	--	--

Información del Proyecto

Empresa / Organización	Altamira Developers
Proyecto	MADES
Fecha de preparación	26/02/2026
Cliente	no hay cliente especificado
Patrocinador principal	Altamira
Gerente / Líder de Proyecto	Yesid Alejandro Martinez guerrero
Gerente / Líder de Desarrollo de Software	Yesid Alejandro Martinez guerrero

Aprobaciones

No aplica

Why are we here?

Se quiere desarrollar una aplicación denominada “MADES” para el manejo y gestión de inventario de un local con actualizaciones de inventario según ventas a tiempo real a nivel general, acceso a un inventario local para acceso a la información en caso de interferencias con el inventario global y registro de las ventas por empleado.

Para entender la idea del desarrollo debemos identificar los problemas generales que enfrentan varios locales de pequeño y mediano tamaño:

- **Descentralización de la información:** Los datos de inventario están registrados tanto en cuadernos y notas físicas como en archivos en dispositivos del negocio, lo que genera la consulta de información sobre el inventario aumente su dificultad por la necesidad de revisar varias fuentes de información, sino que también se pone en riesgo la integridad de dicha información al no poder validar si un registro documentado en un lugar tiene la misma información que el mismo registro documentado en otro lugar.
- **Información no disponible:** Se detectan casos en donde no se puede acceder al inventario del negocio ya sea por errores técnicos en los dispositivos donde se almacena la información, o errores humanos de no saber la ubicación de las anotaciones físicas del inventario.
- **Actualizaciones erróneas de unidades:** Se evidencian errores de índole humana al descontar o agregar unidades en valores en un producto (se descuentan unidades en un producto A cuando se vendió el producto B), y,

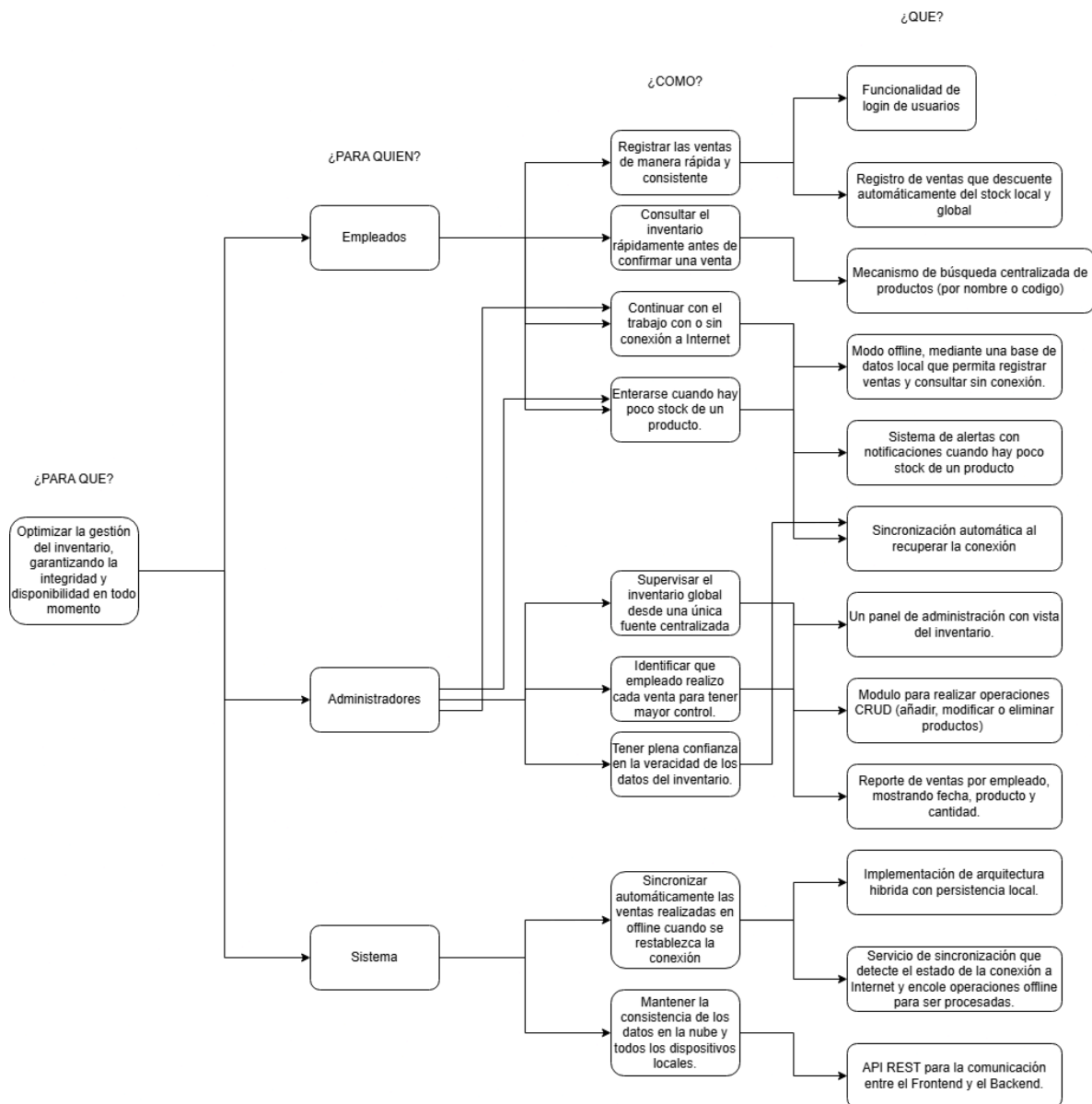
en caso de anotaciones manuales se evidencian anotaciones que por ilegibilidad de la anotación no se logra identificar el producto o es confundido por otro con un nombre o número de identificación parecido, o el valor de unidades disponibles no es claro. Además, de que por errores humanos se evidencian olvidos de actualizaciones de las unidades disponibles al realizar una venta, lo que genera información errónea dentro del inventario.

- **Cobros erróneos:** Se han evidenciado cobros por valores erróneos ya sea cobrar de menos lo que significa pérdidas para el negocio, o costos adicionales que disgustan y molestan a los clientes, esto debido a la desinformación del costo de un producto por parte del empleado.

La solución consiste en el desarrollo de una Aplicación Web Progresiva (PWA) de arquitectura híbrida (Cloud-Native con persistencia local) que permita una gestión y control del inventario más efectiva en donde se producen actualizaciones a tiempo real y en todos los dispositivos para mantener la integridad de la información, búsquedas centralizadas a tiempo real del inventario disponible, disponibilidad de la información y reporte de ventas para mantener un flujo contable óptimo.

Nombre del recurso	Rol
Edison Ferney Gutiérrez Buitrago	Desarrollador front
Johan Sebastián Gil Salamanca	Scrum Master / Desarrollador
Yesid Alejandro Martinez guerreo	Product Owner / Desarrollador
Miguel Leonardo Avila Avila	Desarrollador Back

Impact mapping



Elevator pitch

Para los dueños de negocios y gestores de almacenes que buscan eficiencia, integridad y orden, quienes constantemente se enfrentan a descuadres en su stock que les genera incertidumbres en sus cuentas, y para aquellos que sufren por deficiencia operativa cada vez que falla la conexión a internet, el proyecto MADES se presenta como una solución informática moderna para estos dolores de cabeza.

Es una herramienta diseñada para automatizar el control de existencias mediante lectura de códigos, centralizando toda la información en la nube para que cada

venta se refleje al instante en cualquier dispositivo conectado. A diferencia de los sistemas tradicionales que dependen de una conexión permanente o de procesos manuales propensos al error, nuestro proyecto garantiza la continuidad del negocio gracias a su arquitectura híbrida, permitiendo registrar ventas en modo offline que se sincronizan automáticamente al recuperar la señal, informando siempre con precisión la frescura de los datos locales para que el control del almacén nunca se detenga.

Product Box

Beneficios del proyecto

- Funcionamiento online y offline
- En modo offline, el sistema permite la realización de ventas utilizando datos del almacenamiento local, cuando se restablece la conexión, sincroniza automáticamente los cambios con el inventario general
- El inventario permanece actualizado con fin de ofrecer productos en stock
- Acceso al sistema desde cualquier lugar en modo Online

MADES



“The control makes reality”

Funcionamiento Online y Offline
Sincronización del inventario general
Consistencia en los datos

Create a NOT list

Dentro de alcance	Fuera de alcance
Login mediante usuario registrado	Pagos o transacciones de las ventas
Control de registro de ventas realizadas	Seguimiento contable general del inventariado
Cotización de precios de productos	Sistemas de comunicación entre usuarios de la aplicación
Gestión crud de productos dentro del inventario	Datos de los clientes
Inventario y venta offline	

Sin resolver o decidir

Alcance de una o varias sedes del negocio (si no está en la primera versión se propone como una mejora en versiones posteriores)
En caso de decidir el alcance de varias sedes, realizar una base de datos local que funcione como principal en caso de caída de internet para que en un local se tenga la información de ese local actualizada.
Lectura de códigos de barras mediante cámara

Meet your neighbors

Nombre	Grupo	Información contacto
Edison Ferney Gutiérrez Buitrago	Altamira	322 8369127 / edison.gutierrez01@uptc.edu.co
Johan Sebastián Gil Salamanca	Altamira	312 5406205 / johan.gil@uptc.edu.co
Yesid Alejandro Martínez guerreo	Altamira	3212554009 / yesid.martinez04@uptc.edu.co
Miguel Leonardo Avila Avila	Altamira	320 2510024 / miguel.avila04@uptc.edu.co

Show your solution

Descripción general

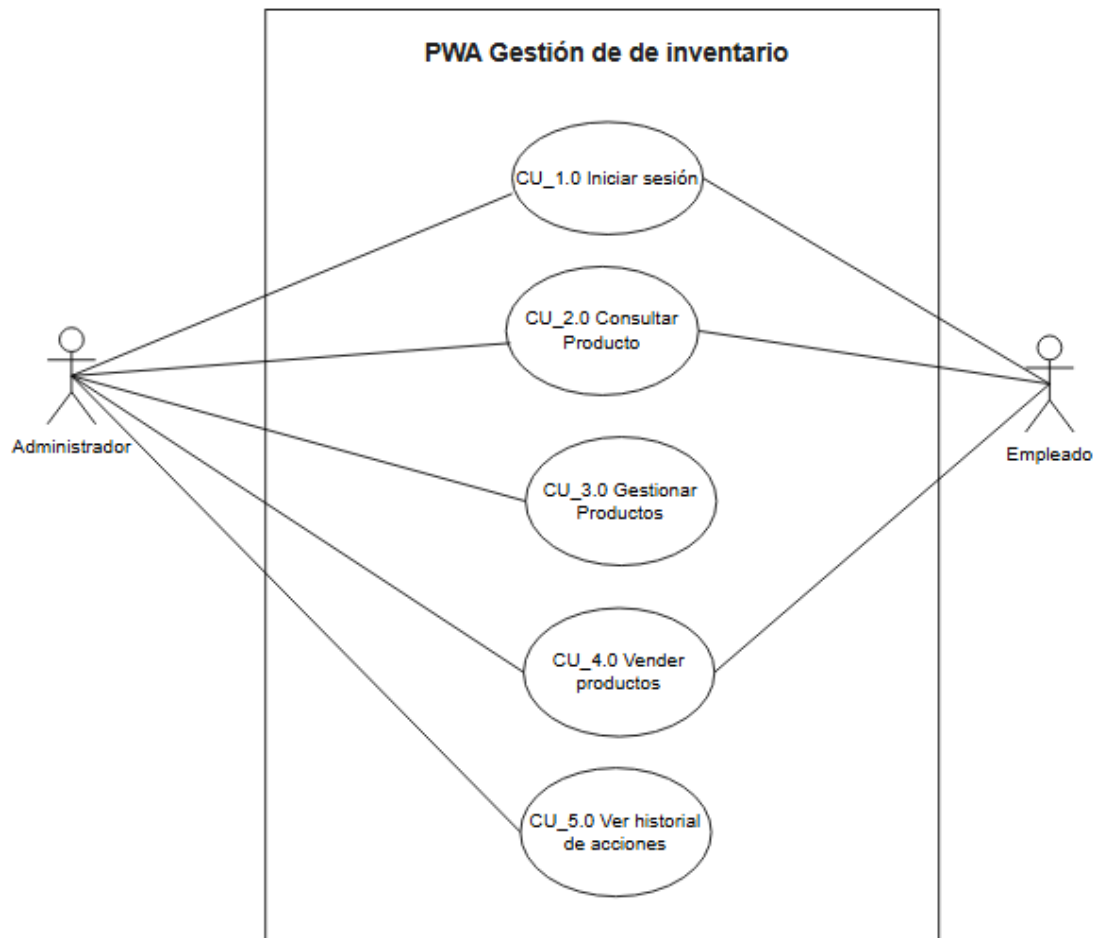
La solución propuesta consiste en una Aplicación Web Progresiva (PWA) orientada a la gestión de inventario de un negocio, permitiendo registrar ventas, consultar productos y mantener sincronizada la información entre los dispositivos. La aplicación funcionará bajo una arquitectura híbrida, teniendo en cuenta:

- Persistencia local (offline): permite garantizar la disponibilidad de la aplicación.
- Sincronización en la nube: permite que la información de los dispositivos que usen la aplicación se encuentre sincronizada.

Monitoreo de Almacén y Datos de Existencias Sincronizados.

Diagrama de casos de uso de contexto

El siguiente diagrama representa visualmente las funcionalidades principales que ofrece la aplicación, tanto para administradores como para empleados:

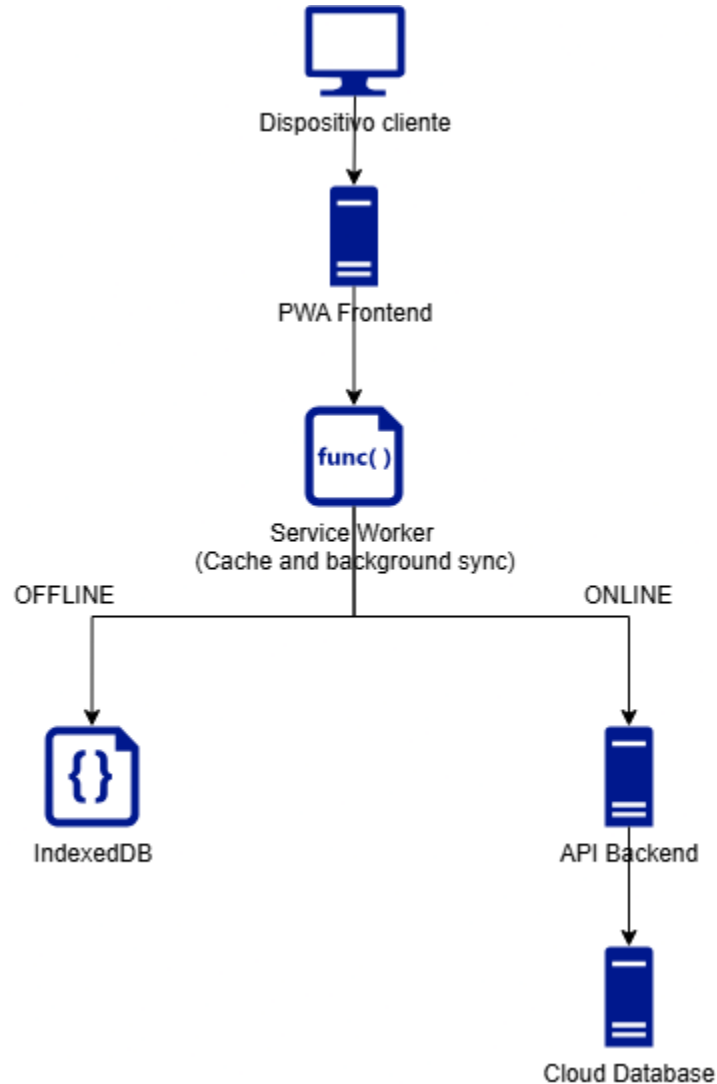


Arquitectura

La arquitectura planteada como híbrida se compone de los siguientes componentes:

- **PWA Frontend:** la aplicación que es instalada en el dispositivo cliente. Utiliza un Service Worker para almacenar los datos localmente permitiendo guardar datos en la base de datos local o sincronizar con la aplicación en la nube.
- **API Backend:** la aplicación en la nube que se conecta con la base de datos en la nube.
- **Base de datos en la nube:** es la que almacena toda la información actual de la aplicación.

Estos componentes se relacionan de la siguiente manera:



Flujo principal de uso

1. Inicio

- El empleado abre la PWA desde cualquier dispositivo.
- Inicia sesión con su usuario correspondiente.
- El sistema sincroniza automáticamente el inventario en la nube.
- Los datos en la nube se descargan en una copia local para uso offline.

2. Registro de una venta

- El empleado busca el producto
- Visualiza la información del producto
- Registra la venta
- El sistema descuenta las unidades del inventario y registra el empleado responsable.
- Guarda la operación localmente.

3. Sincronización automática

- Si hay conexión: la actualización del inventario se realiza en todos los dispositivos.
- Si no hay conexión: la venta se almacena localmente y el sistema intentará constantemente sincronizar cuando vuelva la conexión.

4. Supervisión administrativa

Un usuario administrador puede:

- Ver inventario de productos
- Consultar ventas por empleado
- Ver el historial de ventas de un empleado.

Tecnologías

La solución requiere de las siguientes tecnologías necesarias para la implementación de la arquitectura propuesta:

- **Servidor PWA FrontEnd**

Tecnología seleccionada: Framework Web Angular.

Lenguaje de programación: Typescript.

Justificación: es un framework que permite la construcción de interfaces modernas, modulares y reutilizables. Adicionalmente este framework tiene funciones de PWA, siendo uno de los más utilizados con este propósito gracias a muchas facilidades que ofrece.

- **Servidor Backend**

Tecnología seleccionada: Framework Web SpringBoot.

Lenguaje de programación: Java.

Justificación: es un framework backend muy sólido orientado a soluciones robustas y servicios web escalables. Adicionalmente dispone de varias herramientas ligadas al entorno Spring, que facilitan la implementación de varias funcionalidades como el control de roles y la autenticación con Spring Security.

- **Base de datos en la nube**

Tecnología seleccionada: Base de datos relacional con PostgreSQL.

Lenguaje de consultas: SQL.

Justificación: es un motor que puede ser desplegado fácilmente mediante contenedores Docker, lo que permite ejecutarla de forma aislada y portable en distintos entornos. Además se basa en SQL, que implementa propiedades ACID, valiosas para garantizar consistencia de los datos.

- **Comunicación**

La comunicación entre el servidor PWA y el servidor Backend se realizará mediante el protocolo HTTP/HTTPS, usando servicios REST.

La sincronización en segundo plano se implementará mediante la API Background Sync, proporcionada por los Service Workers de PWA.

Ask what keeps us up at night?

Preocupación /Riesgo	Nivel de impacto
Se quiere realizar un control de movimientos de los usuarios de rol “Empleado” para que aparte de poder ver los cambios y las ventas efectuadas, indirectamente se detecten movimientos sospechosos de los empleados evitando robos internos, pero si bien se prevé poder escribir el motivo de descuento de productos para justificar el descuento que no son por ventas, aún hay posibilidad de pasar movimientos falsos de, ejemplo, producto roto, cuando realmente ha sido robado	4: Bajo: Ese tipo de control de seguridad a un nivel tan exhaustivo no está contemplado dentro del alcance pese a que indirectamente ayuda a evitar que eso mismo suceda.
En un caso offline de larga duración, las actividades que se vayan desarrollando dentro del negocio serán cada vez más inexactas si se perdura mucho tiempo sin conexión	2: Alto: Afecta directamente uno de los puntos fuertes dentro del proyecto ya que está diseñado para periodos no tan largos de no tener acceso a actualizaciones online.
Se puede dar el que se modifique el inventario local de un dispositivo de forma manual, lo que afectará el uso de este en el modo sin conexión	2: Alto: repercute en la veracidad de la información usada por el usuario de dicho inventario local
Si un usuario deja el dispositivo logueado y disponible para cualquier otra persona, esta podría hacer uso del sistema y realizar ventas que físicamente no se efectuaron	1: Muy alto: Es una brecha de seguridad de gran preocupación, aunque, repercute más en el cuidado que tenga el personal que lo use más que en las posibilidades de seguridad de la aplicación.

Size it up

Número de recursos	Rol	Competencias
1	Scrum Master / Líder de Proyecto	Gestión ágil (Scrum), planificación, seguimiento, comunicación con stakeholders
1	Product Owner	Definición de requerimientos, priorización del backlog, conocimiento del negocio
1	Desarrollador Frontend (Angular PWA)	Angular, TypeScript, Service Workers, IndexedDB, UX/UI
1	Desarrollador Backend (Spring Boot)	Java, Spring Boot, REST, Spring Security, JWT, PostgreSQL

Duración estimada: 10 a 12 semanas

Trabajando en Sprints de 2 semanas (Scrum):

Sprint	Entregable Principal
Sprint 1	Configuración de entorno, arquitectura base, autenticación
Sprint 2	CRUD de productos + Base de datos
Sprint 3	Registro de ventas + descuento automático de inventario
Sprint 4	Implementación modo offline (IndexedDB + Service Worker)
Sprint 5	Sincronización automática + pruebas de conflicto
Sprint 6	Reportes por empleado + pruebas finales + despliegue

Complejidad del proyecto

Nivel: Media

Factores que aumentan complejidad:

- Sincronización offline-online
- Manejo de conflictos de datos
- Control de integridad del inventario
- Seguridad (roles y autenticación)

Be clear on what's going to give

Factor	Nivel de Prioridad	Justificación
Alcance	Medio	El alcance puede ajustarse priorizando un MVP que cubra las funcionalidades esenciales: registro de productos, ventas, control de inventario y sincronización offline/online.
Tiempo	Alto	El negocio actualmente presenta problemas operativos que generan pérdidas económicas y desorganización. Por ello, es importante entregar una solución funcional en el menor tiempo posible para empezar a reducir errores manuales, descuadres de inventario y cobros incorrectos.
Costo	Medio	El proyecto busca optimizar recursos utilizando tecnologías open-source como Angular, Spring Boot y PostgreSQL, lo que reduce costos de licenciamiento.
Calidad	Muy Alto	La calidad es el factor más crítico del proyecto, ya que el sistema maneja información sensible para la operación diaria del negocio. Errores en sincronización, pérdida de datos offline o fallos en el descuento de inventario podrían generar impactos financieros directos.

Prioridad Estratégica

1. Calidad (No negociable)

El sistema debe:

- Evitar errores en inventario
- Garantizar sincronización correcta
- No perder datos offline
- Evitar cobros erróneos

Un sistema inestable generaría más problemas que soluciones.

2. Tiempo (Importante)

El negocio necesita dejar el manejo manual lo antes posible.
Por ello se prioriza entregar un **MVP funcional** primero.

3. Alcance (Flexible)

Si hay retrasos, pueden moverse a futuras versiones:

- Multi-sede avanzada
- Lectura automática de código de barras
- Reportes avanzados
- Dashboard analítico

4. Costo (Controlado)

El uso de tecnologías open-source (Angular, Spring Boot, PostgreSQL, Docker) reduce costos de licencia

Show what it's going to take

Costos del proyecto		
Concepto	Coste (Dólares)	Recurso
Equipo de trabajo (diseñador UI, desarrolladores, etc)	\$2000 - \$5000	Programador / empresa de desarrollo
Costos mensuales		
Servidor Web VPS	\$10 - \$25	DigitalOcean, AWS
Base de datos	\$15 - \$30	PostgreSQL, MySQL
Almacenamiento (Copias de seguridad)	\$2 - \$5	AWS S3
Dominio de internet	\$1 - \$2	Hostinger, Cloudflare
Mantenimiento y soporte	\$50 -\$150	Desarrollador equipo de trabajo